

**Five Year Integrated M.Sc. in Computer Science  
(Artificial Intelligence & Data Science)**

**Fifth Semester  
Laboratory Record**

**24-813-0505: Design and Analysis of Algorithms**

*Submitted in partial fulfillment  
of the requirements for the award of a degree in  
Five Year Integrated M.Sc. in Computer Science  
(Artificial Intelligence & Data Science) of  
Cochin University of Science and Technology (CUSAT), Kochi*



*Submitted by:*

**Ahmed Sultan P**  
(Reg No : 24100811)

**DEPARTMENT OF COMPUTER SCIENCE  
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)  
KOCHI - 682022  
November 2026**

**DEPARTMENT OF COMPUTER SCIENCE**  
**COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)**  
**KOCHI, KERALA-682022**



*This is to certify that the software laboratory record for **24-813-0505: Design and Analysis of Algorithms** is a record of work carried out by **Ahmed Sultan P (24100811)** in partial fulfillment of the requirements for the award of degree in **Five Year Integrated M.Sc. in Computer Science (Artificial Intelligence & Data Science)** of Cochin University of Science and Technology (CUSAT), Kochi. The lab record has been approved as it satisfies the academic requirements concerning the Fourth-semester laboratory prescribed for the Five Year Integrated M.Sc. in Computer Science degree.*

**Faculty Member In-Charge :**

**Dr. Shailesh Sivan**  
**Assistant Professor**  
**Department of Computer Science**  
**CUSAT**

**Dr. Bijoy A. Jose**  
**Professor and Head**  
**Department of Computer Science**  
**CUSAT**

## INDEX

| Exp No. | Experiment / Program Title | Page No. |
|---------|----------------------------|----------|
| 1       | Quick Sort                 | 3        |
| 2       | Merge Sort                 | 7        |
| 3       | Heap Sort                  | 11       |
| 4       | Insertion Sort             | 15       |
| 5       | Binary Tree                | 19       |
| 6       | Binary Search Tree (BST)   | 23       |
| 7       | In-order Traversal         | 27       |
| 8       | Pre-order Traversal        | 31       |
| 9       | Post-order Traversal       | 35       |
| 10      | BFS (Breadth-First Search) | 39       |
| 11      | DFS (Depth-First Search)   | 43       |
| 12      | Prim's Algorithm           | 47       |
| 13      | Kruskal's Algorithm        | 51       |
| 14      | Dijkstra's Algorithm       | 55       |
| 15      | Bellman-Ford Algorithm     | 59       |
| 16      | Floyd-Warshall Algorithm   | 63       |



# Experiment 1

## Quick Sort

### QUESTION

---

Write a C++ program to sort a given sequence of elements using the Quick Sort algorithm.

### AIM

---

To implement the Quick Sort algorithm and verify its working on an array of integers.

### THEORY

---

Quick Sort is a highly efficient, divide-and-conquer sorting algorithm. It selects a 'pivot' element and partitions the other elements into two sub-arrays (less than and greater than the pivot). The sub-arrays are then sorted recursively. It is an in-place sort.

### ALGORITHM

---

1. Select the highest index element as the pivot.
2. Partition the array such that elements smaller than the pivot are on the left and larger are on the right.
3. Recursively apply Quick Sort to the left and right sub-arrays.

### COMPLEXITY ANALYSIS

---

**Time:** Best/Avg  $O(N \log N)$ , Worst  $O(N^2)$

**Space:**  $O(\log N)$

## Output

Enter the number of elements: 5

Enter the elements: 30 10 50 20 40

Sorted array using Quick Sort: 10 20 30 40 50

## Program

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int partition(vector<int>& arr, int low, int high) {
6      int pivot = arr[high];
7      int i = (low - 1);
8      for (int j = low; j <= high - 1; j++) {
9          if (arr[j] <= pivot) {
10             i++;
11             swap(arr[i], arr[j]);
12         }
13     }
14     swap(arr[i + 1], arr[high]);
15     return (i + 1);
16 }
17 void quick_sort_helper(vector<int>& arr, int low, int high) {
18     if (low < high) {
19         int pi = partition(arr, low, high);
20         quick_sort_helper(arr, low, pi - 1);
21         quick_sort_helper(arr, pi + 1, high);
22     }
23 }
24 void quick_sort(vector<int>& arr) {
25     if (!arr.empty()) {
26         quick_sort_helper(arr, 0, arr.size() - 1);
27     }
28 }
29 int main() {
30     int size;
31     cout << "Enter the number of elements: ";
32     cin >> size;
33     vector<int> arr(size);
34     cout << "Enter the elements: ";
35     for (int i = 0; i < size; i++) cin >> arr[i];
36
37     quick_sort(arr);
38     cout << "Sorted array using Quick Sort:";
39     for (int i = 0; i < size; i++) cout << " " << arr[i];
40     cout << endl;
41     return 0;
42 }
```



## Experiment 2

### Merge Sort

#### QUESTION

---

Write a C++ program to sort an array of integers using the Merge Sort algorithm.

#### AIM

---

To implement the Merge Sort algorithm and demonstrate its divide-and-conquer approach.

#### THEORY

---

Merge Sort is a stable, divide-and-conquer sorting algorithm. It divides the input array into two halves, recursively sorts the two halves, and then merges the sorted halves. It guarantees  $O(N \log N)$  time complexity but requires extra space.

#### ALGORITHM

---

1. Find the middle point to divide the array into two halves.
2. Recursively call Merge Sort for the first half and the second half.
3. Merge the two sorted halves into a single sorted array using temporary arrays.

#### COMPLEXITY ANALYSIS

---

**Time:** Best/Avg/Worst  $O(N \log N)$

**Space:**  $O(N)$

## Output

Enter the number of elements: 6

Enter the elements: 12 11 13 5 6 7

Sorted array using Merge Sort: 5 6 7 11 12 13

## Program

```
1 #include <iostream>
2 #include <vector>
3 #include <climits>
4 using namespace std;
5
6 void merge(vector<int>& arr, int left, int mid, int right) {
7     int n1 = mid - left + 1;
8     int n2 = right - mid;
9     vector<int> L(n1 + 1), R(n2 + 1);
10    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
11    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
12
13    L[n1] = INT_MAX;
14    R[n2] = INT_MAX;
15    int i = 0, j = 0;
16    for (int k = left; k <= right; k++) {
17        if (L[i] <= R[j]) {
18            arr[k] = L[i];
19            i++;
20        } else {
21            arr[k] = R[j];
22            j++;
23        }
24    }
25 }
26
27 void merge_sort_helper(vector<int>& arr, int left, int right) {
28     if (left < right) {
29         int mid = left + (right - left) / 2;
30         merge_sort_helper(arr, left, mid);
31         merge_sort_helper(arr, mid + 1, right);
32         merge(arr, left, mid, right);
33     }
34 }
35
36 void merge_sort(vector<int>& arr) {
37     if (!arr.empty()) merge_sort_helper(arr, 0, arr.size() - 1);
38 }
39
40 int main() {
41     int size;
42     cout << "Enter the number of elements: ";
43     cin >> size;
44     vector<int> arr(size);
45     cout << "Enter the elements: ";
46     for (int i = 0; i < size; i++) cin >> arr[i];
47
48     merge_sort(arr);
49     cout << "Sorted array using Merge Sort:";
50     for (int i = 0; i < size; i++) cout << " " << arr[i];
51     cout << endl;
52     return 0;
53 }
```



## Experiment 3

### Heap Sort

#### QUESTION

---

Write a C++ program to sort an array using the Heap Sort algorithm.

#### AIM

---

To implement the Heap Sort algorithm using a Max-Heap data structure.

#### THEORY

---

Heap Sort is a comparison-based technique based on a Binary Heap. It builds a Max-Heap from the data, swaps the largest element (root) with the last item, reduces the heap size, and heapifies the new root. This repeats until sorted.

#### ALGORITHM

---

1. Build a Max-Heap from the input array.
2. Swap the root with the last element and reduce the heap size by 1.
3. Call **heapify** on the root to restore the Max-Heap property.
4. Repeat steps 2-3 until all elements are sorted.

#### COMPLEXITY ANALYSIS

---

**Time:** Best/Avg/Worst  $O(N \log N)$

**Space:**  $O(1)$

## Output

Enter the number of elements: 5

Enter the elements: 4 10 3 5 1

Sorted array using Heap Sort: 1 3 4 5 10

## Program

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void heapify(vector<int>& arr, int n, int i) {
6     int largest = i;
7     int left = 2 * i + 1;
8     int right = 2 * i + 2;
9     if (left < n && arr[left] > arr[largest])
10         largest = left;
11     if (right < n && arr[right] > arr[largest])
12         largest = right;
13     if (largest != i) {
14         swap(arr[i], arr[largest]);
15         heapify(arr, n, largest);
16     }
17 }
18 void heap_sort(vector<int>& arr) {
19     int n = arr.size();
20     for (int i = n / 2 - 1; i >= 0; i--)
21         heapify(arr, n, i);
22     for (int i = n - 1; i > 0; i--) {
23         swap(arr[0], arr[i]);
24         heapify(arr, i, 0);
25     }
26 }
27 int main() {
28     int size;
29     cout << "Enter the number of elements: ";
30     cin >> size;
31     vector<int> arr(size);
32     cout << "Enter the elements: ";
33     for (int i = 0; i < size; i++) cin >> arr[i];
34
35     heap_sort(arr);
36     cout << "Sorted array using Heap Sort:";
37     for (int i = 0; i < size; i++) cout << " " << arr[i];
38     cout << endl;
39     return 0;
40 }
```



## Experiment 4

# Insertion Sort

### QUESTION

---

Write a C++ program to sort elements using the Insertion Sort algorithm.

### AIM

---

To sort an array of integers by repeatedly placing each element in its proper position in a sorted subsection.

### THEORY

---

Insertion sort builds the final sorted array one item at a time. It iterates through the input elements, growing a sorted list by comparing the current element to the already sorted ones and inserting it into the correct position.

### ALGORITHM

---

1. Iterate from the second element to the last element of the array.
2. Compare the current element (**key**) with the sorted elements before it.
3. Shift all elements greater than the **key** one position to the right.
4. Insert the **key** into its correct position.

### COMPLEXITY ANALYSIS

---

**Time:** Best  $O(N)$ , Avg/Worst  $O(N^2)$

**Space:**  $O(1)$

## Output

Enter the number of elements: 5

Enter the elements: 12 11 13 5 6

Sorted array using Insertion Sort: 5 6 11 12 13

## Program

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void insertion_sort(vector<int>& arr) {
6      int n = arr.size();
7      for (int i = 1; i < n; i++) {
8          int key = arr[i];
9          int j = i - 1;
10         while (j >= 0 && arr[j] > key) {
11             arr[j + 1] = arr[j];
12             j--;
13         }
14         arr[j + 1] = key;
15     }
16 }
17 int main() {
18     int size;
19     cout << "Enter the number of elements: ";
20     cin >> size;
21     vector<int> arr(size);
22     cout << "Enter the elements: ";
23     for (int i = 0; i < size; i++) cin >> arr[i];
24
25     insertion_sort(arr);
26     cout << "Sorted array using Insertion Sort:";
27     for (int i = 0; i < size; i++) cout << " " << arr[i];
28     cout << endl;
29     return 0;
30 }
```



## Experiment 5

# Binary Tree Construction & Level Order Traversal

### QUESTION

---

Write a C++ program to dynamically build a Binary Tree and perform level-order traversal.

### AIM

---

To implement a Binary Tree using nodes and display its elements using breadth-first traversal.

### THEORY

---

A Binary Tree is a structure in which each node has at most two children. Level-order traversal visits nodes level by level from top to bottom and left to right, typically implemented using a Queue.

### ALGORITHM

---

1. Initialize a queue with the root node.
2. While the queue is not empty, dequeue the front node and print its data.
3. If the left child exists, enqueue it. If the right child exists, enqueue it.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(N)$

**Space:**  $O(N)$

## Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): -1
Enter right child of 2 (-1 for none): -1
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Level Order Traversal: 1 2 3
```

## Program

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  struct Node {
6      int data;
7      Node* left; Node* right;
8      Node(int val) : data(val), left(nullptr), right(nullptr) {}
9  };
10
11 Node* build_binary_tree() {
12     int val;
13     cout << "Enter root value (-1 for no node): ";
14     cin >> val;
15     if (val == -1) return nullptr;
16     Node* root = new Node(val);
17     queue<Node*> q;
18     q.push(root);
19     while (!q.empty()) {
20         Node* curr = q.front(); q.pop();
21         int leftVal, rightVal;
22         cout << "Enter left child of " << curr->data << " (-1 for none): ";
23         cin >> leftVal;
24         if (leftVal != -1) {
25             curr->left = new Node(leftVal);
26             q.push(curr->left);
27         }
28         cout << "Enter right child of " << curr->data << " (-1 for none): ";
29         cin >> rightVal;
30         if (rightVal != -1) {
31             curr->right = new Node(rightVal);
32             q.push(curr->right);
33         }
34     }
35     return root;
36 }
37 void level_order(Node* root) {
38     if (!root) return;
39     queue<Node*> q; q.push(root);
40     cout << "Level Order Traversal:";
41     while (!q.empty()) {
42         Node* curr = q.front(); q.pop();
43         cout << " " << curr->data;
44         if (curr->left) q.push(curr->left);
45         if (curr->right) q.push(curr->right);
46     }
47     cout << endl;
48 }
49 int main() {
50     Node* root = build_binary_tree();
51     level_order(root);
52     return 0;
53 }
```



## Experiment 6

# Binary Search Tree (BST)

### QUESTION

---

Write a C++ program to insert elements into a Binary Search Tree and print them in In-order traversal.

### AIM

---

To implement the insertion logic of a BST and verify its sorting property via In-order traversal.

### THEORY

---

A BST is a binary tree where the left subtree contains keys lesser than the root, and the right subtree contains keys greater than the root. In-order traversal yields the elements in ascending order.

### ALGORITHM

---

1. To insert, check if the root is null. If yes, create a new node.
2. If  $\text{value} < \text{root}$ , recursively insert into the left subtree.
3. If  $\text{value} > \text{root}$ , recursively insert into the right subtree.

### COMPLEXITY ANALYSIS

---

**Time:** Avg  $O(\log N)$ , Worst  $O(N)$

**Space:**  $O(N)$

## Output

Enter the number of elements: 5

Enter the elements: 50 30 70 20 40

In-order Traversal of BST: 20 30 40 50 70

## Program

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     int data;
6     Node* left; Node* right;
7     Node(int val) : data(val), left(nullptr), right(nullptr) {}
8 };
9 Node* insert(Node* root, int val) {
10     if (!root) return new Node(val);
11     if (val < root->data)
12         root->left = insert(root->left, val);
13     else if (val > root->data)
14         root->right = insert(root->right, val);
15     return root;
16 }
17 void inorder(Node* root) {
18     if (!root) return;
19     inorder(root->left);
20     cout << " " << root->data;
21     inorder(root->right);
22 }
23 int main() {
24     int size, val;
25     cout << "Enter the number of elements: ";
26     cin >> size;
27
28     Node* root = nullptr;
29     cout << "Enter the elements: ";
30     for (int i = 0; i < size; i++) {
31         cin >> val;
32         root = insert(root, val);
33     }
34     cout << "In-order Traversal of BST:";
35     inorder(root);
36     cout << endl;
37     return 0;
38 }
```



## Experiment 7

# In-order Traversal

### QUESTION

---

Write a C++ program to build a binary tree and perform In-order traversal.

### AIM

---

To implement the depth-first In-order traversal technique for a Binary Tree.

### THEORY

---

In-order traversal is a depth-first search strategy where the nodes are visited in the order: Left Subtree  $\rightarrow$  Root Node  $\rightarrow$  Right Subtree.

### ALGORITHM

---

1. Build a binary tree level by level using a queue.
2. Traverse the left subtree recursively.
3. Visit (print) the current root node.
4. Traverse the right subtree recursively.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(N)$

**Space:**  $O(N)$

## Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
...
In-Order Traversal: 4 2 5 1 3
```

## Program

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  struct Node {
6      int data;
7      Node* left; Node* right;
8      Node(int val) : data(val), left(nullptr), right(nullptr) {}
9  };
10
11 Node* build_tree() {
12     int val;
13     cout << "Enter root value (-1 for no node): ";
14     cin >> val;
15     if (val == -1) return nullptr;
16     Node* root = new Node(val);
17     queue<Node*> q;
18     q.push(root);
19     while (!q.empty()) {
20         Node* curr = q.front(); q.pop();
21         int leftVal, rightVal;
22         cout << "Enter left child of " << curr->data << " (-1 for none): ";
23         cin >> leftVal;
24         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
25         cout << "Enter right child of " << curr->data << " (-1 for none): ";
26         cin >> rightVal;
27         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
28     }
29     return root;
30 }
31
32 void in_order(Node* root) {
33     if (!root) return;
34     in_order(root->left);
35     cout << " " << root->data;
36     in_order(root->right);
37 }
38
39 int main() {
40     Node* root = build_tree();
41     cout << "In-Order Traversal:";
42     in_order(root);
43     cout << endl;
44     return 0;
45 }
```



## Experiment 8

# Pre-order Traversal

### QUESTION

---

Write a C++ program to construct a binary tree and perform Pre-order traversal.

### AIM

---

To explore and implement the Pre-order traversal of a binary data structure.

### THEORY

---

Pre-order traversal visits nodes in the sequence: Root Node  $\rightarrow$  Left Subtree  $\rightarrow$  Right Subtree. This is often used to copy a tree or evaluate expression trees.

### ALGORITHM

---

1. Build a binary tree level by level.
2. Visit (print) the data of the current root.
3. Recursively traverse the left child.
4. Recursively traverse the right child.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(N)$

**Space:**  $O(N)$

## Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
...
Pre-Order Traversal: 1 2 4 5 3
```

## Program

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  struct Node {
6      int data;
7      Node* left; Node* right;
8      Node(int val) : data(val), left(nullptr), right(nullptr) {}
9  };
10
11 Node* build_tree() {
12     int val;
13     cout << "Enter root value (-1 for no node): ";
14     cin >> val;
15     if (val == -1) return nullptr;
16     Node* root = new Node(val);
17     queue<Node*> q;
18     q.push(root);
19     while (!q.empty()) {
20         Node* curr = q.front(); q.pop();
21         int leftVal, rightVal;
22         cout << "Enter left child of " << curr->data << " (-1 for none): ";
23         cin >> leftVal;
24         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
25         cout << "Enter right child of " << curr->data << " (-1 for none): ";
26         cin >> rightVal;
27         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
28     }
29     return root;
30 }
31
32 void pre_order(Node* root) {
33     if (!root) return;
34     cout << " " << root->data;
35     pre_order(root->left);
36     pre_order(root->right);
37 }
38
39 int main() {
40     Node* root = build_tree();
41     cout << "Pre-Order Traversal:";
42     pre_order(root);
43     cout << endl;
44     return 0;
45 }
```



## Experiment 9

# Post-order Traversal

### QUESTION

---

Write a C++ program to build a binary tree and demonstrate Post-order traversal.

### AIM

---

To implement the Post-order traversal logic on a Binary Tree structure.

### THEORY

---

Post-order traversal visits nodes in the sequence: Left Subtree  $\rightarrow$  Right Subtree  $\rightarrow$  Root Node. It is commonly used when deleting trees.

### ALGORITHM

---

1. Construct a binary tree level by level.
2. Recursively traverse the left subtree.
3. Recursively traverse the right subtree.
4. Visit (print) the data of the current root.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(N)$

**Space:**  $O(N)$

## Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
...
Post-Order Traversal: 4 5 2 3 1
```

## Program

```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4
5 struct Node {
6     int data;
7     Node* left; Node* right;
8     Node(int val) : data(val), left(nullptr), right(nullptr) {}
9 };
10
11 Node* build_tree() {
12     int val;
13     cout << "Enter root value (-1 for no node): ";
14     cin >> val;
15     if (val == -1) return nullptr;
16     Node* root = new Node(val);
17     queue<Node*> q;
18     q.push(root);
19     while (!q.empty()) {
20         Node* curr = q.front(); q.pop();
21         int leftVal, rightVal;
22         cout << "Enter left child of " << curr->data << " (-1 for none): ";
23         cin >> leftVal;
24         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
25         cout << "Enter right child of " << curr->data << " (-1 for none): ";
26         cin >> rightVal;
27         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
28     }
29     return root;
30 }
31
32 void post_order(Node* root) {
33     if (!root) return;
34     post_order(root->left);
35     post_order(root->right);
36     cout << " " << root->data;
37 }
38
39 int main() {
40     Node* root = build_tree();
41     cout << "Post-Order Traversal:";
42     post_order(root);
43     cout << endl;
44     return 0;
45 }
```



## Experiment 10

# Breadth-First Search (BFS)

### QUESTION

---

Write a C++ program to perform Breadth-First Search (BFS) on an adjacency list graph.

### AIM

---

To implement the BFS algorithm to systematically visit graph vertices level-by-level.

### THEORY

---

BFS explores all neighbor nodes at the present depth before moving on to the next depth level. It uses a queue and a visited array to maintain traversal order and avoid cycles.

### ALGORITHM

---

1. Enqueue the starting node and mark it as visited.
2. While queue is not empty, dequeue a node and print it.
3. Iterate adjacent nodes. If unvisited, mark visited and enqueue.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(V + E)$

**Space:**  $O(V)$

## Output

```
Enter the number of nodes: 4
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 1
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): 3
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 3 (Enter -1 to stop): -1
Enter the starting node: 0
0 1 2 3
```

## Program

```
1  #include<iostream>
2  #include<vector>
3  #include<queue>
4  using namespace std;
5
6  void bfs(const vector<vector<int>>& adj, int start, int size) {
7      queue<int> q;
8      q.push(start);
9      vector<bool> is_visited(size, false);
10     while(!q.empty()) {
11         int curr = q.front();
12         cout<<" "<<curr;
13         q.pop();
14         for (int v : adj[curr]) {
15             if (!is_visited[v]) {
16                 is_visited[v]=true;
17                 q.push(v);
18             }
19         }
20     }
21 }
22 int main() {
23     int size, start, v;
24     cout << "Enter the number of nodes: ";
25     cin >> size;
26     vector<vector<int>> adj(size);
27     for (int i = 0; i < size; i++) {
28         do {
29             cout << "Enter Neighbour Vertex of Vertex " << i << " (Enter -1 to stop): ";
30             cin >> v;
31             if (v == -1) break;
32             adj[i].push_back(v);
33         } while (true);
34     }
35     cout << "Enter the starting node: ";
36     cin >> start;
37     bfs(adj, start, size);
38     return 0;
39 }
```



## Experiment 11

# Depth-First Search (DFS)

### QUESTION

---

Write a C++ program to traverse a graph using the Depth-First Search (DFS) algorithm.

### AIM

---

To recursively explore a graph structure utilizing the DFS methodology.

### THEORY

---

DFS explores as far as possible along each branch before backtracking. It relies heavily on recursion (which acts as a stack) and a visited array to prevent infinite loops.

### ALGORITHM

---

1. Define a recursive helper function passing the current node.
2. Mark the current node as visited and print its value.
3. For all adjacent unvisited nodes, recursively call the function.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(V + E)$

**Space:**  $O(V)$

## Output

```
Enter the number of nodes: 4
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 1
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): 3
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 3 (Enter -1 to stop): -1
Enter the starting node: 0
0 1 3 2
```

## Program

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  void dfs_helper(const vector<vector<int>>& adj, int curr, vector<bool>& is_visited) {
6      is_visited[curr] = true;
7      cout << " " << curr;
8      for (int v : adj[curr]) {
9          if (!is_visited[v]) {
10             dfs_helper(adj, v, is_visited);
11         }
12     }
13 }
14 void dfs(const vector<vector<int>>& adj, int start, int size) {
15     vector<bool> is_visited(size, false);
16     dfs_helper(adj, start, is_visited);
17 }
18 int main() {
19     int size, start, v;
20     cout << "Enter the number of nodes: ";
21     cin >> size;
22     vector<vector<int>> adj(size);
23     for (int i = 0; i < size; i++) {
24         do {
25             cout << "Enter Neighbour Vertex of Vertex " << i << " (Enter -1 to stop): ";
26             cin >> v;
27             if (v == -1) break;
28             adj[i].push_back(v);
29         } while (true);
30     }
31     cout << "Enter the starting node: ";
32     cin >> start;
33     dfs(adj, start, size);
34     return 0;
35 }
```



## Experiment 12

# Prim's Algorithm

### QUESTION

---

Write a C++ program to find the Minimum Spanning Tree (MST) of a graph using Prim's algorithm.

### AIM

---

To implement Prim's algorithm using a Priority Queue and calculate the total weight.

### THEORY

---

Prim's algorithm builds an MST by starting from an arbitrary vertex and at each step adding the cheapest possible connection from the tree to another unvisited vertex.

### ALGORITHM

---

1. Push the starting node into a Min-Heap (weight 0, parent -1).
2. Extract the minimum edge. If unvisited, mark visited and add to MST.
3. Push all unvisited adjacent nodes and their weights into the Min-Heap.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(E \log V)$

**Space:**  $O(V + E)$

## Output

```
Enter the number of vertices: 3
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 1 2
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 2 3
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): 2 1
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): 0 2
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 2 (-1,-1 To Stop Reading): -1 -1
Edges in the Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 1
Total Weight of MST: 3
```

## Program

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 using namespace std;
5
6 void run_prims(const vector<vector<pair<int, int>>>& adj, int size) {
7     priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<>> pq;
8     vector<bool> in_mst(size, false);
9     pq.push({0, {0, -1}});
10    int mst_weight = 0;
11    cout << "Edges in the Minimum Spanning Tree:\n";
12
13    while (!pq.empty()) {
14        auto [w, edge] = pq.top(); pq.pop();
15        auto [u, parent] = edge;
16        if (in_mst[u]) continue;
17
18        in_mst[u] = true;
19        if (parent != -1) {
20            cout << parent << " - " << u << " : " << w << endl;
21            mst_weight += w;
22        }
23        for (auto [v, weight] : adj[u]) {
24            if (!in_mst[v]) pq.push({weight, {v, u}});
25        }
26    }
27    cout << "Total Weight of MST: " << mst_weight << endl;
28 }
29
30 int main() {
31     int v, d, size;
32     cout << "Enter the number of vertices: ";
33     cin >> size;
34     vector<vector<pair<int, int>>> adj(size);
35     for (int i = 0; i < size; i++) {
36         do {
37             cout << "Enter Neighbour Vertex Distance of Vertex " << i << " (-1,-1 To Stop): ";
38             cin >> v >> d;
39             if (v == -1 && d == -1) break;
40             adj[i].push_back({v, d});
41         } while (true);
42     }
43     run_prims(adj, size);
44     return 0;
45 }
```



## Experiment 13

# Kruskal's Algorithm

### QUESTION

---

Write a C++ program to find the MST of a given graph using Kruskal's algorithm.

### AIM

---

To implement Kruskal's algorithm using the Disjoint Set (Union-Find) data structure.

### THEORY

---

Kruskal's algorithm sorts edges from low to high weight and adds them to the spanning tree if they don't form a cycle. Cycle detection is managed by a Disjoint Set structure.

### ALGORITHM

---

1. Sort all edges in non-decreasing order of weight.
2. Iterate through sorted edges. Check if vertices belong to different sets using Union-Find.
3. Unify sets and include the edge weight in the total if no cycle is formed.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(E \log E)$

**Space:**  $O(V + E)$

## Output

```
Enter the number of vertices: 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 1 10
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 2 6
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 3 5
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 1 3 15
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 2 3 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): -1 -1 -1
Edges in the Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total Weight of MST: 19
```

## Program

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  struct Edge {
7      int u, v, weight;
8      bool operator<(const Edge& other) const { return weight < other.weight; }
9  };
10 struct DisjointSet {
11     vector<int> parent, rank;
12     DisjointSet(int n) {
13         parent.resize(n); rank.resize(n, 0);
14         for (int i = 0; i < n; i++) parent[i] = i;
15     }
16     int find(int i) {
17         if (parent[i] == i) return i;
18         return parent[i] = find(parent[i]);
19     }
20     bool unite(int i, int j) {
21         int root_i = find(i), root_j = find(j);
22         if (root_i != root_j) {
23             if (rank[root_i] < rank[root_j]) parent[root_i] = root_j;
24             else if (rank[root_i] > rank[root_j]) parent[root_j] = root_i;
25             else { parent[root_j] = root_i; rank[root_i]++; }
26             return true;
27         }
28         return false;
29     }
30 };
31 int main() {
32     int size, u, v, w;
33     cout << "Enter the number of vertices: ";
34     cin >> size;
35     vector<Edge> edges;
36     do {
37         cout << "Enter Edge (u v w) (Enter -1 -1 -1 To Stop): ";
38         cin >> u >> v >> w;
39         if (u == -1 && v == -1 && w == -1) break;
40         edges.push_back({u, v, w});
41     } while (true);
42
43     sort(edges.begin(), edges.end());
44     DisjointSet ds(size);
45     int mst_weight = 0;
46
47     cout << "Edges in the Minimum Spanning Tree:\n";
48     for (const auto& edge : edges) {
49         if (ds.unite(edge.u, edge.v)) {
50             cout << edge.u << " - " << edge.v << " : " << edge.weight << endl;
51             mst_weight += edge.weight;
52         }
53     }
54     cout << "Total Weight of MST: " << mst_weight << endl;
55     return 0;
56 }
```



## Experiment 14

# Dijkstra's Algorithm

### QUESTION

---

Write a C++ program to compute the shortest paths from a source using Dijkstra's algorithm.

### AIM

---

To find the shortest paths in a non-negative weighted graph using a Priority Queue.

### THEORY

---

Dijkstra's algorithm tracks shortest distances from the source and continually relaxes the edges of the minimally distant node until all nodes are processed.

### ALGORITHM

---

1. Initialize distances to infinity, except the source (0). Insert source into Min-Heap.
2. Extract the minimum distance vertex.
3. For adjacent vertices, if `current dist + weight < known dist`, update and push to Heap.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(E \log V)$

**Space:**  $O(V + E)$

## Output

```
Enter the number of vertices: 3
Enter Neighbour Vertex Distance of Vectex 0(Enter -1,-1 To Stop Reading): 1 4
Enter Neighbour Vertex Distance of Vectex 0(Enter -1,-1 To Stop Reading): 2 10
Enter Neighbour Vertex Distance of Vectex 0(Enter -1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vectex 1(Enter -1,-1 To Stop Reading): 2 3
Enter Neighbour Vertex Distance of Vectex 1(Enter -1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vectex 2(Enter -1,-1 To Stop Reading): -1 -1
Distance from Node 0 to Node 0 is : 0
Distance from Node 0 to Node 1 is : 4
Distance from Node 0 to Node 2 is : 7
```

## Program

```
1 #include<iostream>
2 #include<vector>
3 #include<queue>
4 #include<climits>
5 using namespace std;
6
7 vector<int> run_digikstra(const vector<vector<pair<int,int>>>& adj, int start) {
8     vector<int> dist(adj.size(),INT_MAX);
9     dist[start] = 0;
10    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
11    pq.push({0,start});
12    while (!pq.empty()) {
13        auto [d,u] = pq.top(); pq.pop();
14        if (d>dist[u]) continue;
15        for (auto [v,w] : adj[u]) {
16            if (dist[u]+w < dist[v]) {
17                dist[v] = dist[u]+w;
18                pq.push({dist[v],v});
19            }
20        }
21    }
22    return dist;
23 }
24 int main() {
25     int v, d, size;
26     cout<<"Enter the number of vertices: ";
27     cin>>size;
28     vector<vector<pair<int,int>>> adj(size);
29     for (int i=0;i<size;i++) {
30         do {
31             cout<<"Enter Neighbour Vertex Distance of Vectex "<<i<<" (-1,-1 To Stop): ";
32             cin>>v>>d;
33             if (v == -1 && d == -1) break;
34             adj[i].push_back({v,d});
35         } while (true);
36     }
37     vector<int> distances = run_digikstra(adj,0);
38     for (int i=0;i<size;i++) {
39         cout<<"Distance from Node 0 to Node "<<i<<" is : "<<distances[i]<<endl;
40     }
41     return 0;
42 }
```



## Experiment 15

# Bellman-Ford Algorithm

### QUESTION

---

Write a C++ program to find the shortest path utilizing the Bellman-Ford algorithm.

### AIM

---

To implement Bellman-Ford, which inherently supports graphs with negative weight edges.

### THEORY

---

The algorithm relaxes all edges  $V - 1$  times. If distances change in a  $V^{th}$  pass, a negative weight cycle exists.

### ALGORITHM

---

1. Initialize distance from source to infinity, and source to 0.
2. Repeat  $V - 1$  times: if  $\text{dist}[u] + w < \text{dist}[v]$ , update  $\text{dist}[v] = \text{dist}[u] + w$ .

### COMPLEXITY ANALYSIS

---

**Time:**  $O(V \cdot E)$

**Space:**  $O(V)$

## Output

```
Enter the number of vertices: 3
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 1 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 1 2 3
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 2 10
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): -1 -1 -1
Distance from Node 0 to Node 0 is : 0
Distance from Node 0 to Node 1 is : 4
Distance from Node 0 to Node 2 is : 7
```

## Program

```
1 #include <iostream>
2 #include <vector>
3 #include <climits>
4 using namespace std;
5
6 struct Edge {
7     int u, v, weight;
8 };
9
10 vector<int> run_bellman_ford(int size, const vector<Edge>& edges, int start) {
11     vector<int> dist(size, INT_MAX);
12     dist[start] = 0;
13     for (int i = 1; i <= size - 1; i++) {
14         for (const auto& edge : edges) {
15             if (dist[edge.u] != INT_MAX && dist[edge.u] + edge.weight < dist[edge.v]) {
16                 dist[edge.v] = dist[edge.u] + edge.weight;
17             }
18         }
19     }
20     return dist;
21 }
22
23 int main() {
24     int size, u, v, w;
25     cout << "Enter the number of vertices: ";
26     cin >> size;
27     vector<Edge> edges;
28     do {
29         cout << "Enter Edge (u v w) (Enter -1 -1 -1 To Stop): ";
30         cin >> u >> v >> w;
31         if (u == -1 && v == -1 && w == -1) break;
32         edges.push_back({u, v, w});
33     } while (true);
34
35     vector<int> distances = run_bellman_ford(size, edges, 0);
36     for (int i = 0; i < size; i++) {
37         cout << "Distance from Node 0 to Node " << i << " is : " << distances[i] << endl;
38     }
39     return 0;
40 }
```



## Experiment 16

# Floyd-Warshall Algorithm

### QUESTION

---

Write a C++ program to find the all-pairs shortest paths using the Floyd-Warshall algorithm.

### AIM

---

To find the shortest distances between every pair of vertices in a given graph via Dynamic Programming.

### THEORY

---

Floyd-Warshall considers all vertices as intermediate nodes. It iteratively updates the distance between pairs  $(i, j)$  if passing through a node  $k$  yields a shorter path.

### ALGORITHM

---

1. Initialize an  $N \times N$  matrix. Use infinity for missing edges.
2. For intermediate  $k$ , iterate over all  $(i, j)$  pairs.
3. Update  $\text{dist}[i][j]$  if  $\text{dist}[i][k] + \text{dist}[k][j]$  is shorter.

### COMPLEXITY ANALYSIS

---

**Time:**  $O(V^3)$

**Space:**  $O(V^2)$

## Output

Enter the number of vertices: 3

Enter the adjacency matrix (type 'INF' for infinity):

0 3 INF

INF 0 1

INF INF 0

Shortest path matrix:

0 3 4

INF 0 1

INF INF 0

## Program

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <climits>
5 using namespace std;
6
7 void run_floyd_warshall(vector<vector<int>>& dist, int size) {
8     for (int k = 0; k < size; k++) {
9         for (int i = 0; i < size; i++) {
10             for (int j = 0; j < size; j++) {
11                 if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX && dist[i][k] + dist[k][j] < dist[
12                     i][j]) {
13                     dist[i][j] = dist[i][k] + dist[k][j];
14                 }
15             }
16         }
17     }
18 }
19
20 int main() {
21     int size;
22     cout << "Enter the number of vertices: ";
23     cin >> size;
24     vector<vector<int>> dist(size, vector<int>(size));
25     cout << "Enter the adjacency matrix (type 'INF' for infinity):\n";
26     for (int i = 0; i < size; i++) {
27         for (int j = 0; j < size; j++) {
28             string s;
29             cin >> s;
30             if (s == "INF" || s == "inf") dist[i][j] = INT_MAX;
31             else dist[i][j] = stoi(s);
32         }
33     }
34
35     run_floyd_warshall(dist, size);
36
37     cout << "Shortest path matrix:\n";
38     for (int i = 0; i < size; i++) {
39         for (int j = 0; j < size; j++) {
40             if (dist[i][j] == INT_MAX) cout << "INF ";
41             else cout << dist[i][j] << " ";
42         }
43         cout << endl;
44     }
45     return 0;
46 }
```